

# 差分约束

## 人员

王恩泽 到课

## 上周作业检查

上周作业链接: <https://cppoj.kids123code.com/contest/4520>

### 王向东老师周六十点半C++综合练习

刷新

| # | 用户名         | 姓名  | 编程分 | 时间   | A   | B   | C   | D   | E |
|---|-------------|-----|-----|------|-----|-----|-----|-----|---|
| 1 | yujiarui    | 于家瑞 | 400 | 4174 | 100 | 100 | 100 | 100 |   |
| 2 | taohuisheng | 陶汇笙 | 260 | 2399 | 100 | 100 | 60  |     |   |
| 3 | wangenze    | 王恩泽 | 200 | 2171 | 100 | 100 |     |     |   |
| 4 | guoxurui    | 郭栩睿 | 100 | 1061 | 100 | 0   |     |     |   |

## 本周作业

<https://cppoj.kids123code.com/contest/4672> (课上讲了 A ~ D 这些题, 课后作业是 D 题)

## 课堂表现

今天课上讲了 差分约束 这个知识点, 王恩泽同学听课很认真, 吸收的也很好。

## 课堂内容

### 【模板】差分约束

把  $x_i - x_j \leq c$

转化为  $x_i \leq x_j + c$

转化为一条点 j 到点 i 的, 边权为 c 的边即可

设定一个超级源点, 从超级源点往所有点连边, 跑一遍最短路即可, 因为有负边, 所以需要用 SPFA 求最短路

如果图中有负环, 说明无解

```
#include <bits/stdc++.h>

using namespace std;

const int maxn = 5e3 + 5;
const int inf = 0x3f3f3f3f;
struct node {
    int to, value;
```

```

};
vector<node> vec[maxn];
int dis[maxn], cnt[maxn];
bool st[maxn];

int main()
{
    int n, m; cin >> n >> m;
    while (m -- ) {
        int a, b, c; cin >> a >> b >> c;
        vec[b].push_back({a,c});
    }
    for (int i = 1; i <= n; ++i) vec[0].push_back({i,0});

    memset(dis, 0x3f, sizeof(dis));
    queue<int> q; q.push(0); dis[0] = 0; st[0] = true;
    while (!q.empty()) {
        int u = q.front(); q.pop();
        st[u] = false;
        for (node it : vec[u]) {
            if (dis[u]+it.value < dis[it.to]) {
                dis[it.to] = dis[u]+it.value; ++cnt[it.to];
                if (cnt[it.to] == n) { cout << "NO" << endl; return 0; }
                if (!st[it.to]) { q.push(it.to); st[it.to] = true; }
            }
        }
    }

    for (int i = 1; i <= n; ++i) cout << dis[i] << " ";
    cout << endl;
    return 0;
}

```

## 小 K 的农场

按照题目的要求建边, 可以把原问题转化为一个差分约束问题

```

#include <bits/stdc++.h>

using namespace std;

const int maxn = 5e3 + 5;
const int inf = 0x3f3f3f3f;
struct node {
    int to, value;
};
vector<node> vec[maxn];
int dis[maxn], cnt[maxn];
bool st[maxn];

int main()

```

```

{
    int n, m; cin >> n >> m;
    while (m -- ) {
        int op; cin >> op;
        if (op == 1) {
            int a, b, c; cin >> a >> b >> c;
            vec[a].push_back({b,-c});
        } else if (op == 2) {
            int a, b, c; cin >> a >> b >> c;
            vec[b].push_back({a,c});
        } else {
            int a, b; cin >> a >> b;
            vec[a].push_back({b,0}), vec[b].push_back({a,0});
        }
    }
    for (int i = 1; i <= n; ++i) vec[0].push_back({i,0});

    memset(dis, 0x3f, sizeof(dis));
    queue<int> q; q.push(0); dis[0] = 0; st[0] = true;
    while (!q.empty()) {
        int u = q.front(); q.pop();
        st[u] = false;
        for (node it : vec[u]) {
            if (dis[u]+it.value < dis[it.to]) {
                dis[it.to] = dis[u]+it.value; ++cnt[it.to];
                if (cnt[it.to] == n) { cout << "No" << endl; return 0; }
                if (!st[it.to]) { q.push(it.to); st[it.to] = true; }
            }
        }
    }

    cout << "Yes" << endl;
    return 0;
}

```

### [USACO08OCT] Watering Hole G

建一个虚拟点  $n+1$  号点, 如果在一个地方打井可以认为是跟这个  $n+1$  号点连通, 然后跑 kruskal 即可

```

#include <bits/stdc++.h>

using namespace std;

const int maxn = 300 + 5;
int w[maxn];
struct node {
    int a, b, value;
    bool operator < (const node& p) const { return value < p.value; }
};
vector<node> vec;

```

```

int f[maxn];
int fFind(int x) {
    if (f[x] != x) f[x] = fFind(f[x]);
    return f[x];
}

int main()
{
    int n; cin >> n;
    for (int i = 1; i <= n; ++i) {
        int x; cin >> x; vec.push_back({n+1,i,x});
    }

    for (int i = 1; i <= n; ++i) {
        for (int j = 1; j <= n; ++j) {
            int x; cin >> x; vec.push_back({i,j,x});
        }
    }

    sort(vec.begin(), vec.end());
    for (int i = 1; i <= n+1; ++i) f[i] = i;

    int res = 0;
    for (node it : vec) {
        int a = it.a, b = it.b, value = it.value;
        int fa = fFind(a), fb = fFind(b);
        if (fa != fb) f[fa] = fb, res += value;
    }
    cout << res << endl;
    return 0;
}

```

## Transportation

根据 机场 和 港口 是否使用, 分为 4 次最小生成树来做

```

#include <bits/stdc++.h>
#define int long long

using namespace std;

const int maxn = 2e5 + 5;
struct node {
    int from, to, value;
    bool operator < (const node& p) const { return value < p.value; }
};
vector<node> vec;
int X[maxn], Y[maxn], A[maxn], B[maxn], Z[maxn];
int n, m;

int f[maxn];

```

```
int fFind(int x) {
    if (f[x] != x) f[x] = fFind(f[x]);
    return f[x];
}

int kruskal(int flag1, int flag2) {
    for (int i = 1; i <= n+2; ++i) f[i] = i;
    int res = 0, cnt = n-1 + flag1 + flag2;
    for (node it : vec) {
        int a = it.from, b = it.to, c = it.value;
        if (!flag1 && (a==n+1||b==n+1)) continue;
        if (!flag2 && (a==n+2||b==n+2)) continue;
        int fa = fFind(a), fb = fFind(b);
        if (fa != fb) f[fa] = fb, res += c, --cnt;
    }

    if (cnt) return 1e18;
    return res;
}

signed main()
{
    cin >> n >> m;
    for (int i = 1; i <= n; ++i) cin >> X[i], vec.push_back({n+1,i,X[i]});
    for (int i = 1; i <= n; ++i) cin >> Y[i], vec.push_back({n+2,i,Y[i]});
    for (int i = 1; i <= m; ++i) cin >> A[i] >> B[i] >> Z[i],
    vec.push_back({A[i],B[i],Z[i]});
    sort(vec.begin(), vec.end());

    int res = 1e18;
    for (int i = 0; i <= 1; ++i) {
        for (int j = 0; j <= 1; ++j) res = min(res, kruskal(i,j));
    }
    cout << res << endl;
    return 0;
}
```